Overview

This document describes the formalization in the Rocq prover for the following article.
Anonymous: An Information-flow Proof System for Preemptive Kernel Specifications. (under submission)

The formalization as contained in this folder is currently for the assessment of the above article. Please do not distribute the formalization or use it for other purposes.

The formalization mainly includes:

1. a generic model of multi-core, preemptive OS kernels and contextual user programs (in folder `model`)
2. the unwinding conditions condifentiality_u and integrity_u for our kernel model (in `deduct/properties.v`)
3. an information-flow proof system for preemptive kernel specifications (as is proposed in the above-mentioned article), including the soundness proof for this proof system wrt. condifentiality_u and integrity_u (in folder `deduct`)
4. specifications of a synthetic, preemptive OS kernel supporting partitions, with API functions supporting task creation, task deletion, and send and receive operations for message queues and communication channels, and an interrupt handler supporting two-level scheduling of partitions and tasks (in folder `spec`)
5. instantiation of the proof system with the state type of the example kernel (in `verif/state.v`)
6. proof code establishing the satisfaction of condifentiality_u and integrity_u by the example kernel specifications (in folder `verif`)

Paper-to-code Correspondence

The following table shows the correspondence between contents in the article and their mechanized counterpart in the Rocq prover.

Part in Manuscript	**Definition/Result**	**File in Mechanization**	**Entry**
Section 2	*cur_dom*, *cur_tsk*, *sdom*, *upd_cur_dom*, *upd_cur_tsk*	model/language.v	`Fields of the type class State`
Section 2	Syntax for specification statement	model/language.v	`Inductive spec_code`
Section 2	Semantic judgment (1)	model/opsem.v	`Inductive hpstep`
Section 3	$(P, O) \vdash cfg \xrightarrow{\alpha,d}_{\mathrm{D}} cfg'$	model/opsem.v	`Definition hpstep_d`
Section 3	*confidentiality_u*	deduct/properties.v	`Definition confidentiality_u`
Section 3	*integrity_u*	deduct/properties.v	`Definition integrity_u`
Section 3	$cfg_1 \overset{d}{\sim} cfg_2$	deduct/properties.v	`Definition obsr_equiv`
Section 3	$cfg_1 \overset{\mathbb{S}}{\sim} cfg_2$	deduct/properties.v	`Definition obsr_equiv`
Section 4	proof rules in Fig. 1	deduct/rules.v	`Inductive deduct_rules: …`
Section 4	top-level proof rule	deduct/rules.v	`Inductive top_rule: …`
Section 4	*Theorem 1*	deduct/soundness.v	`Theorem soundness_confidentiality` `Theorem soundness_integrity`
Section 5	*Theorem 2*	verif/verif_result.v	`Theorem sys_sec`

Building and Running the Proof Scripts

This formalization is developed using Rocq 8.13.2

To build the proof scripts, first ensure that the “bin” folder of the Coq installation is in the system path.

A makefile should be generated from _CoqProject using the following command:

```
coq_makefile -f _CoqProject -o Makefile
```

To build the development, issue the “`make`" command in the top-level folder.

After the proof scripts are built, they can be run using Proof General, CoqIDE, or via a VS Code plugin for Coq.

References:

The content of the folders auxlibs and tactics are mostly based on the verification framework CSL-R in

Fengwei Xu, Ming Fu, Xinyu Feng, Xiaoran Zhang, Hui Zhang, and Zhaohui Li. A Practical Verification Framework for Preemptive OS Kernels. In Proceedings of 28th International Conference on Computer Aided Verification (CAV), pp. 59-79, 2016.